

Nightshift

An AI on-call engineer for Kubernetes, built from scratch: from your first FastAPI endpoint to multi-agent incident response, GitOps and LLMOps. Free, on your laptop.

How to use this guide. Read the blueprint once. Then work through the lessons with the starter repository (nightshift.zip): every lesson explains the concepts, walks you through the build step by step, and ends with a definition of done and interview questions. The tests in the repository tell you when each step is finished.

This edition contains the blueprint and lessons M0–M3. Later lessons are written when you reach them.

M0–M3

Foundations: setup, FastAPI, Docker, PostgreSQL

M8–M9

MLOps anomaly detection · RAG with pgvector

M4–M7

Operate it: observability, CI, Kubernetes, GitOps

M10–M13

Agents · multi-agent LangGraph · LLMOps · production

Contents

Nightshift — blueprint

1. Why this project
2. Architecture (the finished system)
3. The free stack
4. Staying within the rules
5. The modules
6. Repository layout (when finished)
7. How to work through a module
8. Final checklist (M13)
9. Results you'll put on your resume
10. Resume bullets (fill in the numbers)
11. Interview story (60 seconds)

M0 • Engineering setup

Concepts

Step 1 • Install the tools

Step 2 • Create the repository

Step 3 • The Python project

Step 4 • Configure ruff and pytest

Step 5 • .gitignore

Step 6 • Makefile

Step 7 • pre-commit

Step 8 • Your first TDD module: money

Step 9 • Push to GitHub

Definition of done

Interview questions

M1 • HTTP APIs with FastAPI

Concepts

Step 1 • Dependencies

Step 2 • Errors (given)

Step 3 • Models: tinyshop/models.py

Step 4 • The repository: tinyshop/repository.py

Step 5 • The API: tinyshop/api.py

Step 6 • Run it: tinyshop/main.py

Definition of done

Interview questions

M2 • Docker

Concepts

Step 1 · Install and warm up

Step 2 · A naive Dockerfile first (to see the problems)

Step 3 · .dockerignore

Step 4 · The production Dockerfile

Step 5 · Compose: API + PostgreSQL

Definition of done

Interview questions

M3 • PostgreSQL

Concepts

Step 1 · SQL warm-up in psql

Step 2 · The first migration

Step 3 · psycopg in five minutes

Step 4 · The migration runner: tinyshop/db.py

Step 5 · The PostgreSQL repository: tinyshop/pg_repository.py

Step 6 · Wire it up: tinyshop/main.py

Step 7 · Test against the real database

Step 8 · Run the whole stack

Definition of done

Interview questions

Nightshift — blueprint

An AI on-call engineer for Kubernetes. When an alert fires at 3 a.m., Nightshift investigates it like a senior SRE would: it checks the metrics and logs, looks at what changed recently, searches the runbooks and past incidents, names the most likely root cause with evidence, proposes a fix for a human to approve, and writes the postmortem.

You build everything yourself, from an empty folder to a GitOps-managed Kubernetes deployment. You also build **TinyShop**, the small online shop that Nightshift operates, so you have a real system to break, observe and fix.

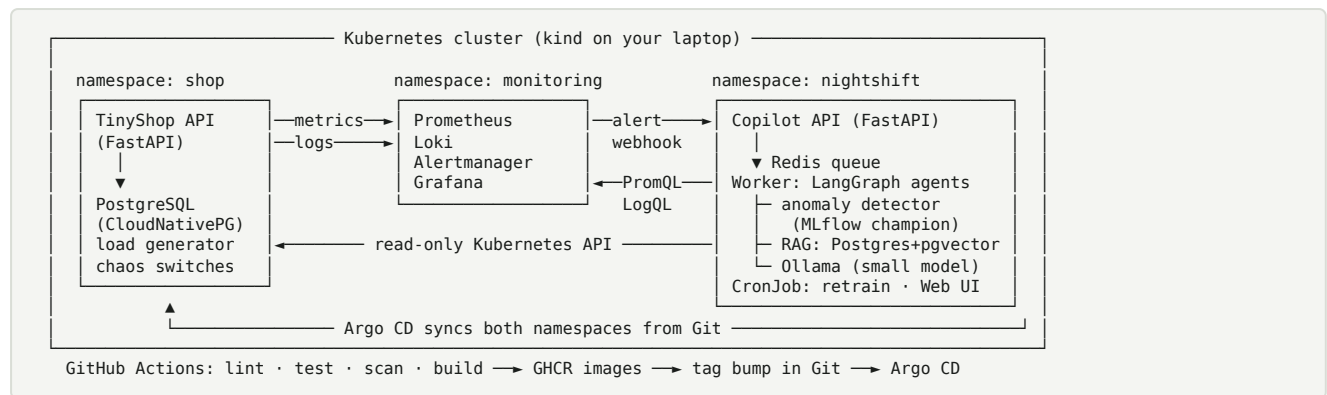
Starting point: you know Python. Everything else (HTTP APIs, SQL, Docker, Kubernetes, CI/CD, MLOps, LLMOps) is taught module by module. Cost: €0. Everything runs on your laptop; public hosting is optional (see "Free production").

1. Why this project

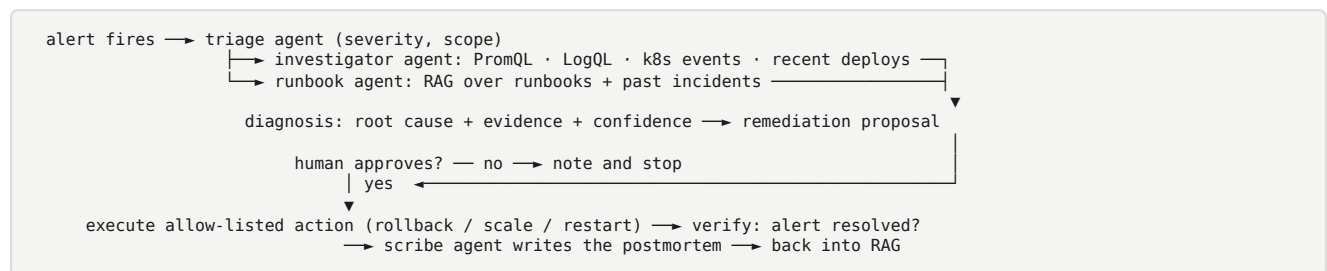
What employers ask for	Where you prove it here
Python services, REST APIs	TinyShop and the copilot API (FastAPI, Pydantic)
SQL and PostgreSQL	schema design, migrations, transactions, row locking, pgvector
Docker	multi-stage images, Compose, non-root, health checks
Kubernetes	Deployments, Services, probes, HPA, ConfigMaps/Secrets, RBAC, operators, Helm
CI/CD and GitOps	GitHub Actions, GHCR, image scanning, Argo CD, staging → prod, rollback
Observability / SRE	Prometheus, Grafana, Loki, Alertmanager, SLOs, runbooks, chaos testing
MLOps	dataset from real metrics, anomaly detection, MLflow tracking and registry, scheduled retraining, drift
RAG	runbooks, postmortems and Kubernetes docs in pgvector, hybrid search, retrieval evals
Agents and multi-agents	tool-calling agents with read-only cluster tools, LangGraph supervisor, human-in-the-loop
LLMOps	tracing, prompt versioning, evals on injected incidents, guardrails, cost and latency tracking

What makes it unique: you can *measure* it. Because you inject every failure yourself (a bad deploy, a slow database, a memory leak), you know the true root cause, so you can report numbers like "Nightshift found the correct root cause in 7 of 8 injected incidents, with a median time to diagnosis of 52 s". Most AI portfolio projects can't measure anything.

2. Architecture (the finished system)



The incident flow:



3. The free stack

Layer	Choice	Why
Language, packaging	Python 3.12, uv , ruff, pytest, pre-commit	fast, modern, standard
Web framework	FastAPI + Pydantic v2	the de-facto Python API framework
Database	PostgreSQL 16 with psycopg 3 and plain SQL migrations; pgvector for RAG	learn real SQL; one database for app data and vectors
Containers	Docker , Docker Compose	local dev stack
Kubernetes	kind (Kubernetes in Docker), kubectl, Helm , k9s	a real cluster on your laptop, free
Postgres on Kubernetes	CloudNativePG operator	learn operators and CRDs; production-grade
Metrics, alerts, dashboards	kube-prometheus-stack (Prometheus, Alertmanager, Grafana)	industry standard
Logs	Loki + Grafana Alloy (Promtail's successor; Promtail reached end of life in March 2026)	logs you can query from code
CI	GitHub Actions , images in GHCR , Trivy scanning	free for public repositories
CD	Argo CD (GitOps)	the most asked-for CD tool in Kubernetes jobs
ML	scikit-learn (IsolationForest + a statistical baseline), MLflow	explainable, CPU-only, fast
LLM	Ollama in the cluster or on your host: <code>llama3.2:3b</code> or <code>qwen3:4b</code> (tool calling), <code>nomic-embed-text</code>	free, private, no API keys
Agents	LangGraph	explicit graphs, state, human-in-the-loop
Queue	Redis + RQ	simple background jobs
LLM tracing	Langfuse (free Hobby cloud tier, or self-hosted)	traces, prompt versions, evals
Load and chaos	Locust; chaos switches built into TinyShop; <code>kubectl</code> fault injection	you control every failure

FREE PRODUCTION

- **Default:** your kind cluster, managed by Argo CD with separate `staging` and `prod` namespaces, is your production-like environment. CI/CD, GitOps, rollbacks and alerting all work exactly as in a company.
- **Optional public cluster:** Oracle Cloud's Always Free tier can run k3s on an ARM VM. Since June 2026 the free ARM allowance is **2 OCPUs and 12 GB RAM** (it was 4 and 24). That's enough for

TinyShop, the copilot and a 3B model on CPU, but slow. Sign-up needs a card for verification, and free ARM capacity is often unavailable. Treat it as a bonus, not a dependency.

- **Demo for recruiters:** a recorded 3-minute video plus the Grafana and incident screenshots in the README. It's free and never goes down.

4. Staying within the rules

This project is designed so that nothing in it breaks a law, a licence or a terms of service:

- **Data is yours.** All metrics, logs and incidents come from TinyShop, which you run and break yourself. TinyShop has no customer accounts, so there is no personal data, and GDPR has nothing to apply to. Never point Nightshift at systems you don't own or aren't allowed to operate.
- **Chaos only in your own cluster.** Fault injection is done through switches in your own app and your own `kubectl`, never against third-party services.
- **Documents for RAG have open licences.** You write the runbooks yourself. The Kubernetes documentation is licensed **CC BY 4.0** (attribution required: keep the source URL on every chunk and credit "The Kubernetes Authors"). Check the licence file of any other source before you ingest it; if there isn't a clear licence, don't use it.
- **Model licences.** Llama 3.2 is under the Llama 3.2 Community License (fine for this; include "Built with Llama" if you distribute). Qwen3 models are Apache 2.0. `nomic-embed-text` is Apache 2.0.
- **Free tiers used as intended.** Public GitHub repositories get free Actions minutes and GHCR storage. Langfuse's Hobby tier is for exactly this. Only synthetic data goes to any hosted service.
- **Safe agents.** Agents get **read-only** Kubernetes permissions (RBAC `get/list/watch`). Actions that change anything (rollback, scale, restart) are allow-listed, need explicit human approval, and are logged. This is also what the OWASP Top 10 for LLM applications calls "excessive agency" protection, and a great interview topic.
- **Secrets never in Git.** `.env` is git-ignored from day one; in Kubernetes, secrets are encrypted before they go to Git (Sealed Secrets, or SOPS + age as a fallback).
- **EU AI Act.** An IT-operations assistant is not a high-risk use case under Annex III. You still label AI-generated diagnoses as AI-generated and keep a human in the loop, which is good practice and good interview material.

5. The modules

Each module has a lesson in `docs/modules/`, tests in `tests/`, a **definition of done** (something real that runs), and interview questions. Pace: 2–3 hours a day, about 16 weeks. Lessons M0–M3 are included now; each next lesson is written when you reach it, so it matches the current versions of the tools.

PART A — FOUNDATIONS: BUILD AND RUN TINYSHOP

#	Module	You learn	You build	Done when
M0	Engineering setup	terminal, Git/GitHub, uv, pytest, TDD, ruff, pre-commit, Make	the repository and <code>tinyshop/money.py</code>	<code>make check M=00</code> green, pushed to GitHub
M1	HTTP APIs with FastAPI	HTTP, REST, JSON, status codes, Pydantic validation, dependency injection, OpenAPI	TinyShop API with an in-memory repository	<code>make check M=01</code> green; you place an order in Swagger UI
M2	Docker	images vs containers, layers and caching, multi-stage builds, non-root, health checks, Compose, volumes, networks	Dockerfile, <code>.dockerignore</code> , <code>compose.yaml</code> with api + db	<code>docker compose up</code> serves TinyShop
M3	PostgreSQL	SQL, schema design, constraints, indexes, migrations, transactions, isolation, row locks, <code>EXPLAIN</code>	migrations, a Postgres repository, race-condition-safe orders	<code>make test-db</code> green; orders survive restarts

PART B — OPERATE IT LIKE A COMPANY WOULD

#	Module	You learn	You build	Done when
M4	Observability and chaos	metrics types, RED/USE, Prometheus, PromQL, structured JSON logs, Loki, LogQL, Grafana, SLIs/SLOs	<code>/metrics</code> , JSON logging with request ids, dashboards, chaos switches (errors, latency, slow queries, memory leak, crash), a Locust load generator	you break TinyShop and see it in Grafana
M5	CI with GitHub Actions	workflows, jobs, services, caching, matrix, secrets, GHCR, image scanning	CI: lint → tests (with a Postgres service) → build → Trivy scan → push image	green pipeline on every push; image in GHCR
M6	Kubernetes	Pods, Deployments, ReplicaSets, Services, Ingress, ConfigMaps, Secrets, probes, requests/limits, HPA, namespaces, operators and CRDs	TinyShop on kind; Postgres via CloudNativePG; kube-prometheus-stack; Loki + Alloy	<code>kubectl</code> shows everything healthy; an alert fires when you inject a fault
M7	Helm and GitOps	Helm charts and values, Argo CD applications, environments, promotion, rollback, drift	a Helm chart; Argo CD managing <code>staging</code> and <code>prod</code> ; CI bumps the image tag in Git	merging to <code>main</code> deploys to staging; a PR promotes to prod; one command rolls back

PART C — THE COPILOT

#	Module	You learn	You build	Done when
M8	MLOps: anomaly detection	datasets from time series, feature engineering, baselines, IsolationForest, precision/recall on labelled incidents, MLflow tracking and registry, scheduled retraining, drift (PSI)	a detector trained on normal traffic, evaluated on injected incidents, served from the MLflow registry, retrained by a Kubernetes CronJob	detector beats the baseline; CronJob promotes only if better
M9	RAG	embeddings, chunking, pgvector, hybrid search, reranking, retrieval evals	runbooks you write, past postmortems, selected Kubernetes docs (CC BY 4.0, with attribution) in pgvector	recall@5 measured on your eval set
M10	Tool-using agent	tool calling, ReAct, structured output, tool safety, timeouts, loop limits	an investigator agent with read-only tools: PromQL, LogQL, Kubernetes events and rollout history	it explains an injected incident from real evidence
M11	Multi-agent system	LangGraph state, supervisor pattern, parallel branches, human-in-the-loop, retries	triage → investigator + runbook → diagnosis → proposal → approval → action → verify → postmortem; Alertmanager webhook; Redis queue; a small web UI	an alert becomes a full incident report without you typing anything
M12	LLMOps	tracing, prompt versioning, eval sets, LLM-as-judge vs exact checks, guardrails, cost and latency budgets, regression gates	Langfuse tracing; an eval harness that injects each chaos scenario and scores root-cause accuracy; a CI gate	a prompt change that lowers accuracy fails CI
M13	Production hardening and launch	least-privilege RBAC, NetworkPolicies, secrets management, resource tuning, load testing, SLOs and error budgets, runbooks, incident review	hardened manifests, encrypted secrets, load-test report, SLO dashboard, final README with results, demo video	everything in the checklist below

CHAOS SCENARIOS (YOUR EVAL SET FROM M4 ONWARDS)

#	You inject	What should be found
1	bad deploy: new version returns 500 on /orders	error rate up right after a rollout → roll back
2	slow queries (missing index, or pg_sleep)	p95 latency up, DB time up → add the index / fix the query
3	memory leak	memory climbs, pod OOMKilled, restarts → fix the leak / raise the limit temporarily
4	CPU limit too low	throttling, latency up under load → raise limits or scale
5	broken config (bad DATABASE_URL)	CrashLoopBackOff, connection errors in logs → fix the ConfigMap/Secret
6	database down	/ready failing, 503s → restore the database
7	traffic spike	saturation, HPA at max → scale out
8	noisy but harmless blip	nothing to do → don't page (tests false positives)

6. Repository layout (when finished)

nightshift/	
tinystore/	the shop you operate (M1–M4)
migrations/	SQL migrations (M3)
copilot/	Nightshift: api, agents, tools, rag, detector (M8–M12)
ml/	training and evaluation scripts (M8)
deploy/	
compose/	local stack (M2–M4)
helm/	Helm charts (M7)
argocd/	Argo CD applications (M7)
k8s/	cluster add-ons, RBAC, NetworkPolicies (M6, M13)
runbooks/	runbooks you write (M9)
evals/	chaos scenarios and retrieval questions (M9, M12)
loadtest/	Locust (M4)
.github/workflows/	CI and CD (M5, M7, M12)
docs/	blueprint, lessons, runbook, model card, results
tests/	one test file per module

7. How to work through a module

1. Read the lesson in docs/modules/. Read the concept parts properly: they're your interview answers.
2. Build step by step. Run `make check M=<nn>` often; red tests tell you what's missing.
3. Do the **definition of done**: run the real thing, not just the tests.
4. Commit in small steps (`git commit -m "M1: place_order"`) and push.
5. Answer the interview questions at the end out loud, without looking.

6. Stuck? Ask for a **hint** first, then the concept with a small example, and only then the solution for one function.

8. Final checklist (M13)

- ☐ `git clone` + one command brings up the whole stack on a fresh machine (README tested by someone else).
- ☐ CI: lint, tests, image scan and eval gate on every PR; images tagged with their git SHA.
- ☐ Argo CD manages staging and prod; a rollback takes one command and under 2 minutes.
- ☐ Every Deployment has probes, requests/limits, a non-root security context and a NetworkPolicy.
- ☐ The agents' ServiceAccount can only `get/list/watch`; the action runner can only do its allow-listed actions.
- ☐ SLO dashboard for TinyShop (availability, p95 latency) with an error budget.
- ☐ Results table filled in (below), with how you measured each number.
- ☐ A 3-minute demo video: inject a fault → alert → diagnosis → approve rollback → resolved → postmortem.

9. Results you'll put on your resume

Metric	How it's measured	Result
Root-cause accuracy	8 chaos scenarios × 5 runs each, exact match on cause category	__ / 40
Median time to diagnosis	alert received → diagnosis written	__ s
False-page rate	scenario 8 (harmless blip)	__ / 5
Anomaly detector	precision / recall vs statistical baseline on labelled windows	__ / __ vs __ / __
Retrieval	recall@5 on runbook questions	__
Delivery	commit to staging (CI + Argo CD)	__ min
Cost per incident	tokens × price (or GPU-seconds for local models)	__

10. Resume bullets (fill in the numbers)

- Built **Nightshift**, an AI on-call copilot for Kubernetes: a LangGraph multi-agent system that investigates Prometheus alerts with read-only PromQL, LogQL and Kubernetes tools plus RAG over runbooks and past incidents, and proposes human-approved remediations; correct root cause in __/40 injected incidents, median diagnosis __ s.

- Operated the target service end to end: FastAPI + PostgreSQL with race-safe transactions, Docker multi-stage images, Kubernetes (kind, CloudNativePG, HPA), Helm and **Argo CD GitOps** with staging/prod promotion and one-command rollback.
- Built CI/CD with GitHub Actions (tests against real Postgres, Trivy image scanning, GHCR) and an **LLM evaluation gate** that blocks prompt or model changes that reduce diagnosis accuracy.
- Trained and deployed an anomaly detector on live metrics with MLflow tracking, registry-based promotion and CronJob retraining; improved precision from __ (baseline) to __.
- Hardened for production: least-privilege RBAC for agents, NetworkPolicies, encrypted secrets, SLOs with error budgets, load-tested to __ requests/s.

11. Interview story (60 seconds)

"I wanted a project where the AI part is measurable. So I built a small shop service, deployed it on Kubernetes with GitOps and full observability, and wrote a chaos toolkit that injects eight kinds of failures. Then I built Nightshift, a multi-agent copilot that gets the Prometheus alert, queries metrics, logs and the Kubernetes API with read-only tools, searches runbooks and past incidents, and proposes a fix that a human approves. Because I inject every failure, I know the true root cause, so I can measure accuracy: it's __ of 40. Every prompt change runs through that eval in CI, so I can't accidentally make it worse."

M0 • Engineering setup

Time: 3–4 sessions • **Tests:** `make check M=00` • **You'll have:** a professional Python repository on GitHub and your first test-driven module.

Most tutorials skip this part. Companies don't: every repository you'll work in has a dependency manager, a lock file, a linter, tests, pre-commit hooks and a Makefile. Setting them up yourself, once, is how you understand them.

Concepts

The terminal. Everything in DevOps happens in a terminal: Docker, kubectl, Git, CI logs. You need about ten commands:

Command	Does	Example
<code>pwd</code>	print the folder you're in	<code>pwd</code>
<code>ls -la</code>	list files, including hidden ones (<code>.gitignore</code> , <code>.env</code>)	<code>ls -la</code>
<code>cd</code>	change folder (<code>..</code> = up one, <code>~</code> = home)	<code>cd ~/code/nightshift</code>
<code>mkdir -p</code>	create folders	<code>mkdir -p tinystop</code>
<code>cat</code>	print a file	<code>cat pyproject.toml</code>
<code>echo \$VAR</code>	print an environment variable	<code>echo \$PATH</code>
<code>export VAR=x</code>	set an environment variable for this terminal	<code>export DATABASE_URL=...</code>
<code>which</code>	where is this program?	<code>which python</code>
<code>Ctrl+C</code>	stop the running program	
<code>↑ / Ctrl+R</code>	previous commands / search history	

Environment variables are how every tool in this course gets configuration: `DATABASE_URL`, `OLLAMA_HOST`, `KUBECONFIG`. `PATH` is the list of folders the shell searches for programs; "command not found" usually means a folder is missing from `PATH`.

Git records snapshots of your project. Four places matter: your *working tree* (files on disk), the *staging area* (`git add` puts changes there), the *history* (`git commit` saves a snapshot) and the *remote* (GitHub; `git push` uploads). Branches are movable labels on commits; later you'll work on a branch, open a pull request and let CI check it before merging.

uv manages Python itself, your virtual environment (`.venv/`, a private copy of Python with your packages) and your dependencies. `pyproject.toml` lists what you *want* ("fastapi, version 0.115 or newer"); `uv.lock` records exactly what you *got* (one exact fastapi version and every sub-dependency, with hashes). Commit both. The lock file is why your laptop, CI and the Docker image all run identical versions.

Tests and TDD. pytest finds files named `test_*.py` and runs functions named `test_*`; a test passes if no `assert` fails. In test-driven development you write (or here: are given) the test first, watch it fail (red), write the simplest code that passes (green), then clean up (refactor).

Linting and formatting. ruff checks for bugs and bad patterns (unused imports, undefined names, mutable default arguments) and formats code consistently, so code reviews are about logic, not spaces.

pre-commit runs checks automatically every time you `git commit`, so bad code never reaches GitHub.

Make gives every project the same short commands (`make test`, `make lint`). Nobody remembers `uv run pytest -q tests/test_m01_*.py`; everybody remembers `make check M=01`.

Step 1 • Install the tools

1. **A Unix terminal.** – Windows: install WSL2 with Ubuntu (`wsl --install` in PowerShell as administrator, then restart). Do **everything** in this course inside Ubuntu, and keep your code in the Linux home folder (`~/code`), not under `/mnt/c`, which is much slower. – Mac: the built-in Terminal. Install Homebrew (<https://brew.sh>) for the other tools. – Linux: you're set.
2. **Git:** `sudo apt install git make` (Ubuntu/WSL) or `brew install git` (Mac; `make` is included with the Xcode command-line tools: `xcode-select --install`).
3. **uv:** follow <https://docs.astral.sh/uv/getting-started/installation/> (a one-line installer). Then `uv -version`.
4. **VS Code** with the extensions *Python*, *Ruff*, and on Windows WSL. On Windows, open the project with `code .` from the Ubuntu terminal.
5. **A GitHub account** and the GitHub CLI (`sudo apt install gh` or `brew install gh`), then `gh auth login`.

Tell Git who you are (this appears on every commit):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Step 2 • Create the repository

Unzip the starter (it only contains `docs/`, `tests/` and a `README`), then:

```
mkdir -p ~/code && cd ~/code
unzip ~/Downloads/nightshift.zip      # adjust the path
cd nightshift
git init
git add . && git commit -m "Starter: docs and tests"
```

Step 3 • The Python project

```
uv init --bare --name nightshift --python 3.12
uv add --dev pytest httpx ruff pre-commit pyyaml
```

- `uv init --bare` creates only `pyproject.toml`. Open it and read it.
- `uv add --dev ...` adds *development* tools: needed to test and lint, but not in production. `uv` writes them under `[dependency-groups]`, creates `.venv/`, installs everything, and writes `uv.lock`.
- Look at what happened: `cat pyproject.toml`, `ls -la`, `head -30 uv.lock`.

Now the package that will hold TinyShop:

```
mkdir tinyshop
```

Create `tinyshop/__init__.py`:

```
"""TinyShop: the small online shop that Nightshift will operate."""

__version__ = "0.1.0"
```

A folder with an `__init__.py` is a Python *package*: `import tinyshop` now works. `__version__` follows *semantic versioning*: MAJOR.MINOR.PATCH.

Step 4 • Configure ruff and pytest

Add to the end of `pyproject.toml`:

```
[tool.ruff]
line-length = 110
target-version = "py312"

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP"] # pycodestyle, pyflakes, import sorting, bugbear, pyupgrade

[tool.ruff.lint.isort]
known-first-party = ["tinyshop", "tests"]

[tool.pytest.ini_options]
testpaths = ["tests"]
markers = ["postgres: needs a running PostgreSQL (TEST_DATABASE_URL)"]
```

Step 5 • .gitignore

Create `.gitignore`:

```
.venv/  
__pycache__/  
*.pyc  
.env  
.pytest_cache/  
.ruff_cache/
```

`.env` will hold passwords later. Ignoring it **before** it exists is how you make sure a secret never lands in Git history, where deleting the file doesn't remove it.

Step 6 • Makefile

Create `Makefile`. **Recipe lines must start with a TAB character, not spaces.** VS Code shows the difference in the bottom-right corner ("Spaces: 4" vs "Tab Size: 4"); if `make` says `missing separator`, that's the cause.

```
.PHONY: install test lint fmt check run  
  
install:          ## install dependencies and git hooks  
    uv sync  
    uv run pre-commit install  
  
test:             ## all tests  
    uv run pytest -q  
  
lint:             ## check style and bugs  
    uv run ruff check .  
    uv run ruff format --check .  
  
fmt:              ## fix what can be fixed automatically  
    uv run ruff check --fix .  
    uv run ruff format .  
  
check:            ## one module's tests: make check M=01  
    uv run pytest -q tests/test_m$(M)_*.py  
  
run:              ## start TinyShop with auto-reload (from M1)  
    uv run uvicorn tinyshop.main:app --reload
```

- A *target* (`test:`) is a name; the indented lines below it are shell commands.
- `$(M)` is a variable you pass on the command line: `make check M=00`.
- `.PHONY` says these targets are commands, not files.
- `uv run X` runs `X` inside your project's virtual environment, so you never need to "activate" it.

Step 7 • pre-commit

Create `.pre-commit-config.yaml`:

```
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.16.9
  hooks:
    - id: ruff
      args: [--fix]
    - id: ruff-format
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v6.0.0
  hooks:
    - id: end-of-file-fixer
    - id: trailing-whitespace
    - id: check-yaml
    - id: detect-private-key
```

Then:

```
make install          # installs the git hook
uv run pre-commit autoupdate  # bumps `rev:` to the newest versions
uv run pre-commit run --all-files  # run all hooks once on everything
```

From now on, `git commit` runs these hooks first. If a hook changes a file, the commit stops: look at the change, `git add` it, and commit again. `detect-private-key` blocks commits that contain private keys, a small but real safety net.

Run the setup checks:

```
make check M=00
```

`test_m00_setup.py` should now pass; `test_m00_money.py` fails because `tinyshop/money.py` doesn't exist yet. That's your next step.

Step 8 • Your first TDD module: money

Why integer cents? Try this in `uv run python`:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Floats are binary fractions and can't represent most decimal amounts exactly. Small errors add up across thousands of orders, and totals stop matching. Every serious payment system stores money as integers in the smallest unit (cents), or as `Decimal`. `TinyShop` uses cents everywhere: `price_cents = 1299` means €12.99.

Open `tests/test_m00_money.py` and read it: the tests are the specification. Then create `tinychop/money.py` with these three functions:

```
def to_cents(amount: str) -> int:
    """Convert a text amount in euros to integer cents.

    "19.99" -> 1999, "19,99" -> 1999 (German comma), "5" -> 500, "0.5" -> 50, " 7.10 " -> 710.
    Surrounding spaces are allowed. At most 2 decimals. Raise ValueError for anything else:
    "", "abc", "1.999", "-5", "1.2.3", "€5".
    Never convert through float.
    """

def format_eur(cents: int) -> str:
    """1999 -> "19.99 EUR", 5 -> "0.05 EUR", 0 -> "0.00 EUR". Negative -> ValueError."""

def split_evenly(cents: int, parts: int) -> list[int]:
    """Split an amount into `parts` shares that differ by at most 1 cent and sum EXACTLY to
    `cents`.
    The first shares get the extra cents: split_evenly(1000, 3) -> [334, 333, 333].
    parts <= 0 -> ValueError."""
```

Work in the TDD loop: run `make check M=00`, read the first failure, make it pass, repeat.

Hints (open only if stuck)

- `to_cents`: strip, then either a regular expression like `^(\d+)(?:[.](\d{1,2}))?$` (Python's `re` module), or split on `.`/`,` and check each part with `str.isdigit()`. Watch out for "0.5": the fraction "5" means 50 cents, not 5. `str.ljust(2, "0")` pads "5" to "50".
- `format_eur`: `cents // 100` and `cents % 100`, and an f-string with `:02d` to pad to two digits.
- `split_evenly`: `divmod(cents, parts)` gives the base share and the remainder; the first remainder shares get one extra cent.

Step 9 • Push to GitHub

```
make lint          # fix anything it reports (make fmt fixes most)
make test          # M0 tests green; M1+ tests fail for now, which is expected
git add .
git commit -m "M0: project setup and money helpers"
gh repo create nightshift --public --source . --push
```

A **public** repository gives you free CI minutes (M5) and free image hosting (GHCR) later, and recruiters can see it. That's also why secrets never go into it.

Definition of done

- ☐ `make check M=00` is green.
- ☐ `make lint` is clean, and `git commit` runs the pre-commit hooks.

- ☐ The repository is on GitHub with `pyproject.toml`, `uv.lock`, `.gitignore`, `Makefile`, `.pre-commit-config.yaml` and `tinychop/`.
- ☐ `git log --oneline` shows small, meaningful commits.

Interview questions

1. **What's the difference between `pyproject.toml` and `uv.lock`?** The first declares the dependencies you want, with version ranges; the lock file pins the exact resolved versions of everything (including sub-dependencies), so every environment installs identical packages. Commit both.
2. **Why is floating point wrong for money? What do you use instead?** Binary floats can't represent most decimals exactly, so rounding errors accumulate. Use integer minor units (cents) or `Decimal`.
3. **What happens if you commit a password and delete it in the next commit?** It stays in Git history and anyone with the repository can read it. Rotate the secret immediately; rewriting history is a clean-up step, not a fix. Prevent it with `.gitignore` and hooks like `detect-private-key`.
4. **What is TDD, and when is it most useful?** Write a failing test, make it pass, refactor. Most useful when the behaviour is clear and edge cases matter, like money, parsing or permissions.
5. **What's a virtual environment, and why use one?** A per-project Python environment with its own packages, so projects don't break each other and you can reproduce the exact setup.

M1 • HTTP APIs with FastAPI

Time: ~1 week · **Tests:** make check M=01 · **You'll have:** TinyShop, a working shop API with products and orders, documented in Swagger UI, with an in-memory store.

TinyShop is the system Nightshift will operate. It's deliberately small, but it has the things real services have: validation, error codes, a data layer behind an interface, and health endpoints for Kubernetes.

Concepts

HTTP IN FIVE MINUTES

A client sends a **request**: a method, a path, headers and sometimes a body. The server returns a **response**: a status code, headers and usually a body. APIs almost always use JSON bodies.

```
POST /orders HTTP/1.1
Content-Type: application/json

{"items": [{"product_id": 1, "quantity": 2}]}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id": 7, "items": [...], "total_cents": 2598, "status": "created", "created_at": "..."}

```

Method	Meaning	Safe?	Idempotent?
GET	read	yes	yes
POST	create / trigger an action	no	no
PUT	replace	no	yes
PATCH	partial update	no	not necessarily
DELETE	delete	no	yes

Idempotent means doing it twice has the same effect as doing it once. It matters for retries: a client may safely retry a GET after a timeout, but retrying a POST can create two orders. (Real shops solve that with idempotency keys; a good interview topic.)

Status codes you'll use:

Code	Meaning	TinyShop example
200 OK	success	GET a product
201 Created	something new was created	POST /products, POST /orders
404 Not Found	the resource doesn't exist	GET /products/999
409 Conflict	valid request, but conflicts with current state	duplicate SKU, out of stock, cancelling twice
422 Unprocessable Content	the body or parameters are invalid	negative price, empty order
500 Internal Server Error	a bug on the server	never on purpose
503 Service Unavailable	a dependency is down	/ready when the database is unreachable

4xx means the client must change something; 5xx means the server has a problem. Alerting in M4 is built on this difference: a spike in 5xx pages someone, a spike in 404s usually doesn't.

REST means organising the API around *resources* (nouns: /products, /orders/7) and using methods for verbs. Actions that don't fit (cancel) become sub-resources: POST /orders/7/cancel.

PYDANTIC: VALIDATION AT THE EDGE

Never trust input. Pydantic models declare what valid data looks like, and FastAPI rejects everything else with a 422 *before* your code runs:

```
from pydantic import BaseModel, Field, field_validator

class BookIn(BaseModel):
    isbn: str = Field(pattern=r"^\d{13}$") # regex the whole value must match
    title: str = Field(min_length=1, max_length=200)
    price_cents: int = Field(gt=0) # greater than 0
    tags: list[str] = Field(default=[], max_length=5) # on a list: at most 5 items

    @field_validator("tags")
    @classmethod
    def lower_case_tags(cls, tags: list[str]) -> list[str]:
        if any(t != t.lower() for t in tags):
            raise ValueError("tags must be lower case") # becomes a 422
        return tags

class Book(BookIn): # inheritance: everything from BookIn, plus an id
    id: int

book = Book(isbn="9780000000000", title="SRE", price_cents=3999)
book.model_dump() # -> dict
Book(id=1, **book.model_dump())
book.model_copy(update={"price_cents": 2999}) # models are treated as immutable values: copy
with changes
```

FASTAPI: A SMALL EXAMPLE

This is not TinyShop; it shows every feature you need so you can write TinyShop yourself.

```
from typing import Annotated
from fastapi import Depends, FastAPI, Query, Request
from fastapi.responses import JSONResponse

class BookNotFound(Exception):
    pass

class BookStore:
    # a tiny "repository"
    def __init__(self):
        self.books: dict[int, Book] = {}
    def get(self, book_id: int) -> Book:
        if book_id not in self.books:
            raise BookNotFound(f"book {book_id} not found")
        return self.books[book_id]

def get_store(request: Request) -> BookStore:
    # a dependency: FastAPI calls it per request
    return request.app.state.store

StoreDep = Annotated[BookStore, Depends(get_store)]

def create_app(store: BookStore) -> FastAPI:
    # an "app factory"
    app = FastAPI(title="Books")
    app.state.store = store
    # objects the whole app shares

    @app.exception_handler(BookNotFound)
    # domain error -> HTTP response, in ONE place
    async def not_found(request: Request, exc: BookNotFound):
        return JSONResponse(status_code=404, content={"detail": str(exc)})

    @app.get("/books/{book_id}")
    # path parameter, converted to int and
    # validated
    def get_book(book_id: int, store: StoreDep) -> Book:
        # return type = response model
        return store.get(book_id)

    @app.get("/books")
    # query parameters: /books?limit=10&offset=20
    def list_books(store: StoreDep,
                   limit: Annotated[int, Query(ge=1, le=100)] = 50,
                   offset: Annotated[int, Query(ge=0)] = 0) -> list[Book]:
        return list(store.books.values())[offset: offset + limit]

    @app.post("/books", status_code=201)
    # a Pydantic parameter = the JSON body
    def add_book(data: BookIn, store: StoreDep) -> Book:
        book = Book(id=len(store.books) + 1, **data.model_dump())
        store.books[book.id] = book
        return book

    return app
```

- **The app factory** (`create_app(store)`) is what makes testing easy: tests pass a fresh store, production passes the real one.
- **Dependency injection** (`Depends`) means endpoints *ask* for what they need instead of creating it. In M3 you'll swap the in-memory store for PostgreSQL without touching a single endpoint.
- **Exception handlers** keep endpoints clean: the repository raises a domain error, one handler turns it into the right status code.
- **Parameters without defaults come first** in Python, so `store: StoreDep` goes before `limit=50`.

- FastAPI generates **OpenAPI** documentation from all of this: open `/docs` in a browser.

THE REPOSITORY PATTERN

The API talks to storage only through an interface (a `Protocol`: "any object with these methods"). In M1 you implement it with Python dicts; in M3 with PostgreSQL. The same tests run against both, which is how you know the database version behaves correctly. This is called a **contract test**; see

`tests/repo_contract.py`.

LIVENESS VS READINESS

Kubernetes (M6) asks every container two questions:

- **Liveness** (`/health`): is the process alive? If not, restart it. It must *not* check the database, or a database outage would make Kubernetes restart every app pod in a loop.
- **Readiness** (`/ready`): can it serve traffic right now? If not, stop sending requests to it until it recovers. This one *does* check dependencies.

Step 1 • Dependencies

```
uv add fastapi "uvicorn[standard]"
```

FastAPI is the framework; uvicorn is the server that runs it.

Step 2 • Errors (given)

Create `tinyshop/errors.py`:

```
class ShopError(Exception):
    """Base class for domain errors."""

class NotFoundError(ShopError):
    pass

class OutOfStockError(ShopError):
    pass

class ConflictError(ShopError):
    pass
```

Step 3 • Models: `tinyshop/models.py`

Write these Pydantic models (the tests in `tests/test_m01_models.py` show every rule):

Model	Fields and rules
ProductIn	sku: str (3–20 chars of A-Z, 0-9, - ; use a regex pattern), name: str (1–100 chars), price_cents: int (> 0), stock: int (≥ 0)
Product(ProductIn)	adds id: int
OrderItemIn	product_id: int (> 0), quantity: int (1–100)
OrderIn	items: list[OrderItemIn], 1 to 20 items, no product twice (a field_validator)
OrderItem	product_id: int, quantity: int, unit_price_cents: int (the price at the time of ordering)
Order	id: int, items: list[OrderItem], total_cents: int, status: Literal["created", "cancelled"], created_at: datetime

Literal["created", "cancelled"] (from typing) means "only these two strings". Why does OrderItem store unit_price_cents? Because prices change. An order must keep the price the customer actually paid.

Step 4 • The repository: `tinychop/repository.py`

Start the file with the interface (given):

```
from __future__ import annotations

from datetime import UTC, datetime
from typing import Protocol

from tinychop.errors import ConflictError, NotFoundError, OutOfStockError
from tinychop.models import Order, OrderIn, OrderItem, Product, ProductIn

class Repository(Protocol):
    def add_product(self, data: ProductIn) -> Product: ...
    def get_product(self, product_id: int) -> Product: ...
    def list_products(self, limit: int = 50, offset: int = 0) -> list[Product]: ...
    def place_order(self, data: OrderIn) -> Order: ...
    def get_order(self, order_id: int) -> Order: ...
    def cancel_order(self, order_id: int) -> Order: ...
    def top_products(self, limit: int = 5) -> list[tuple[str, int]]: ...
    def ping(self) -> None: ...
```

Then write class `InMemoryRepository` with the same methods. The behaviour (every point is tested in `tests/repo_contract.py`):

- **add_product** gives each product a new id (1, 2, 3, ...). A SKU that already exists raises `ConflictError`.
- **get_product** / **get_order**: unknown id raises `NotFoundError`.
- **list_products** returns products ordered by id, skipping `offset` and returning at most `limit`.
- **place_order**:
 - unknown product → `NotFoundError`;
 - not enough stock for **any** item → `OutOfStockError`, and **no stock changes at all** (all or nothing);

- otherwise reduce stock, record each item with the product's *current* price, compute `total_cents`, status `"created"`, `created_at = datetime.now(UTC)`.
- `cancel_order`: puts the stock back and sets status `"cancelled"`. Cancelling an already-cancelled order raises `ConflictError` (and must not add stock twice).
- `top_products(limit)`: (`sku`, `revenue_cents`) for products in **non-cancelled** orders, highest revenue first, then by SKU; at most `limit` rows.
- `ping`: does nothing and returns `None` (the in-memory store is always reachable).

Hints

- Store products and orders in two dicts keyed by id, plus two counters for the next ids.
- All or nothing: first loop over the items and *check* everything (existence and stock); only if every check passes, loop again and *apply* the changes. In M3, a database transaction gives you this for free.
- Pydantic models are values: to change stock, build a copy with `product.model_copy(update={"stock": new_stock})` and store the copy.
- `zip(products, data.items, strict=True)` walks two lists together.
- For `top_products`, add up revenue per product id in a dict, turn it into a list of tuples, and sort with `key=lambda r: (-r[1], r[0])`.

Run `make check M=01`: the repository tests (`TestInMemoryRepository`) should pass before you start the API.

Step 5 • The API: `tinychop/api.py`

Write `create_app(repo: Repository) -> FastAPI` with `title="TinyShop"`, `version=__version__` and these endpoints:

Method and path	Success	Errors
GET /health	200 {"status": "ok", "version": "0.1.0"}	-
GET /ready	200 {"status": "ready"} after <code>repo.ping()</code> succeeds	503 {"status": "unavailable"} if <code>ping()</code> raises anything
POST /products	201 + the product	422 invalid, 409 duplicate SKU
GET /products?limit=50&offset=0	200 + list (limit 1-100, offset ≥ 0)	422 invalid parameters
GET /products/{product_id}	200 + product	404
POST /orders	201 + order	422 invalid, 404 unknown product, 409 out of stock
GET /orders/{order_id}	200 + order	404
POST /orders/{order_id}/cancel	200 + cancelled order	404, 409 already cancelled
GET /stats/top-products?limit=5	200 + [{"sku": ..., "revenue_cents": ...}] (limit 1-50)	422

Error responses have the shape `{"detail": "<message>"}`. Map `NotFoundError` → 404 and both `OutOfStockError` and `ConflictError` → 409 with exception handlers. Use the Books example above as your pattern.

Step 6 • Run it: `tinyshop/main.py`

```
from tinyshop.api import create_app
from tinyshop.repository import InMemoryRepository

app = create_app(InMemoryRepository())
```

(In M3 this becomes "PostgreSQL if `DATABASE_URL` is set, otherwise in-memory".)

```
make run
```

Open <http://127.0.0.1:8000/docs>. This is Swagger UI, generated from your code. Try the whole flow by hand: create two products, place an order, try to order more than the stock (409), cancel the order, check the stock is back, look at `/stats/top-products`.

The same from the terminal with curl, which you'll use a lot in Docker and Kubernetes:

```
curl -s localhost:8000/health
curl -s -X POST localhost:8000/products -H 'content-type: application/json' \
  -d '{"sku":"MUG-001","name":"Mug","price_cents":1299,"stock":10}'
curl -s -X POST localhost:8000/orders -H 'content-type: application/json' \
  -d '{"items":[{"product_id":1,"quantity":2}]}'
curl -s -i localhost:8000/products/999      # -i shows the status line and headers
```

Change a file while `make run` is running: `--reload` restarts the server automatically.

Definition of done

- ☐ `make check M=01` is green (models, repository contract, API).
- ☐ You placed and cancelled an order in Swagger UI and with curl.
- ☐ `make lint` is clean; committed and pushed (`git commit -m "M1: TinyShop API"`).

Interview questions

1. **401 vs 403 vs 404 vs 409 vs 422?** 401: not authenticated. 403: authenticated but not allowed. 404: doesn't exist. 409: valid request that conflicts with the current state. 422: the request itself is invalid.
2. **Which HTTP methods are idempotent, and why does it matter?** GET, PUT, DELETE (and HEAD, OPTIONS). Clients and proxies retry on timeouts; retrying a non-idempotent POST can duplicate an order. Idempotency keys make POSTs safe to retry.
3. **Why an app factory and dependency injection?** Tests can build an app with fakes; production wires real dependencies; endpoints don't know which storage they use.
4. **Where should input validation happen?** At the boundary (request models), so invalid data never reaches business logic, plus database constraints as the last line of defence (M3).
5. **Liveness vs readiness?** Liveness: is the process alive (restart if not); never check dependencies there. Readiness: can it serve now (stop routing traffic if not); check dependencies.
6. **How do you keep an order "all or nothing"?** Validate everything before changing anything; in a database, wrap it in a transaction.

M2 • Docker

Time: ~1 week · **Tests:** `make check M=02` · **You'll have:** TinyShop as a small, secure container image, plus a Compose stack with the API and a PostgreSQL database (which the app starts using in M3).

"It works on my machine" stops being an excuse with containers: the image contains the exact Python, packages and code, and runs the same on your laptop, in CI and in Kubernetes. Every later module builds on this one.

Concepts

Term	Meaning	Analogy
Image	a read-only template: filesystem + metadata (what to run, which port)	a class
Container	a running (or stopped) instance of an image, with its own writable layer	an object
Dockerfile	the recipe that builds an image, one instruction per line	source code
Layer	each instruction adds a layer; layers are cached and shared	git commits
Registry	where images are stored: Docker Hub, GHCR (M5)	GitHub for images
Tag	a name for a version: <code>python:3.12-slim</code> , <code>tinychop:0.1.0</code>	a git tag
Volume	storage that outlives containers (database files)	an external drive

A container is not a virtual machine. It's a normal Linux process that the kernel isolates (namespaces: its own filesystem view, network, process list) and limits (cgroups: CPU and memory). That's why containers start in milliseconds, and why Kubernetes can run many of them per machine. On Mac and Windows, Docker Desktop runs a small Linux VM in the background for you.

Layer caching is the most important performance idea. Docker reuses a cached layer if the instruction *and everything before it* are unchanged. So you order instructions from "changes rarely" to "changes often": base image → dependencies → your code. Then editing `api.py` rebuilds in seconds instead of reinstalling every package.

Multi-stage builds use one stage with build tools to prepare the app and a second, clean stage that only copies the result. The final image doesn't contain the build tools, so it's smaller and has less to attack.

Never run as root. By default, a process in a container runs as root. If someone exploits your app, a root process makes escaping the container or damaging mounted files much easier. Create a user and switch to it. Kubernetes can enforce this (M13).

Configuration comes from the environment, not from the image: the same image runs in dev, staging and prod with different environment variables (the *Twelve-Factor App* principle). Secrets are never baked into an image: anyone who can pull the image can read every layer.

Compose describes several containers that belong together (API + database), their network, volumes and start order, in one YAML file. On the Compose network, containers reach each other **by service name**: from the `api` container, the database host is `db`, not `localhost`. Inside a container, `localhost` is the container itself.

YAML in 30 seconds. Indentation (spaces only, never tabs) means nesting; `key: value` is a mapping; `- item` is a list entry. Quote strings that contain `:` or start with special characters. You'll write a lot of YAML: Compose, GitHub Actions, Kubernetes, Helm.

Step 1 • Install and warm up

Install **Docker Desktop** (Windows: enable "Use WSL 2 based engine" and WSL integration for your Ubuntu; Mac: the Apple-silicon or Intel build) or **Docker Engine** on Linux. Check: `docker version` and `docker compose version`.

Warm up, and read what each command prints:

```
docker run hello-world                # pull an image, run it, exit
docker run -it --rm python:3.12-slim python  # an interactive Python in a container; exit()
docker run -d --name web -p 8080:80 nginx:1.27 # -d background, -p host:container port
curl -s localhost:8080 | head -5
docker ps                             # running containers
docker logs web                       # its output
docker exec -it web sh                # a shell INSIDE the container; try ls /, whoami,
exit
docker stop web && docker rm web
docker images                         # images on your machine and their sizes
```

Step 2 • A naive Dockerfile first (to see the problems)

Create `Dockerfile`:

```
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install fastapi "uvicorn[standard]"
CMD ["uvicorn", "tinyshop.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
docker build -t tinyshop:naive .
docker run --rm -p 8000:8000 tinyshop:naive # open http://localhost:8000/docs, then Ctrl+C
docker images tinyshop                     # note the SIZE
docker run --rm tinyshop:naive whoami      # root!
docker run --rm tinyshop:naive ls -a       # .venv, .git ... all copied in
```

Now change one line in `tinyshop/api.py` and build again: `pip` reinstalls everything, because `COPY . .` came before `pip install`. Problems to fix:

1. **Huge** (the full `python:3.12` image is around 1 GB).
2. **Runs as root**.

3. **Copies everything**, including `.venv`, `.git` and, one day, `.env` with passwords.
4. **No caching** of dependencies.
5. **Ignores `uv.lock`**, so versions can differ from what you tested.

Step 3 • `.dockerignore`

The *build context* is everything Docker sends to the builder (the `.` in `docker build .`). Keep junk and secrets out of it. Create `.dockerignore`:

```
.venv
.git
.env
__pycache__
.pytest_cache
.ruff_cache
tests
docs
```

Step 4 • The production Dockerfile

Replace the `Dockerfile` with this. Read every comment; you'll be asked about these choices in interviews.

```
# ---- build stage: install dependencies with uv ----
FROM python:3.12-slim AS builder
# copy the uv binary from uv's official image; use the version `uv --version` shows
COPY --from=ghcr.io/astral-sh/uv:0.12.19 /uv /bin/uv
# compile .pyc files now (faster start); copy files instead of hard-linking; use the image's Python
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy UV_PYTHON_DOWNLOADS=0
WORKDIR /app
# 1) only the dependency files: this layer is cached until pyproject.toml or uv.lock change
COPY pyproject.toml uv.lock ./
# 2) install EXACTLY the locked versions (--frozen), without pytest/ruff (--no-dev), into /app/.venv
RUN uv sync --frozen --no-dev --no-install-project
# 3) your code last: it changes most often
COPY tinyshop ./tinyshop

# ---- runtime stage: small image, non-root user ----
FROM python:3.12-slim
RUN useradd --create-home --uid 10001 app
WORKDIR /app
COPY --from=builder /app /app
# use the virtual environment's python/uvicorn; print logs immediately (no buffering)
ENV PATH="/app/.venv/bin:$PATH" PYTHONUNBUFFERED=1
USER app
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
# exec form (a JSON list): uvicorn runs as PID 1 and receives the stop signal directly
CMD ["uvicorn", "tinyshop.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why `--host 0.0.0.0`? `127.0.0.1` means "only this container itself", so the port mapping from your laptop couldn't reach it. `0.0.0.0` means "every network interface of the container".

Why the exec form `CMD ["uvicorn", ...]` and not `CMD uvicorn ...`? The shell form starts `/bin/sh -c`, which doesn't forward `SIGTERM`, so `docker stop` (and Kubernetes) would wait 10 seconds and then kill your app instead of letting it shut down cleanly.

Build and compare:

```
docker build -t tinyshop:0.1.0 .
docker images tinyshop           # compare with :naive
docker run --rm tinyshop:0.1.0 whoami # app
docker run -d --name shop -p 8000:8000 tinyshop:0.1.0
sleep 15 && docker ps             # STATUS shows (healthy)
curl -s localhost:8000/health
docker stop shop && docker rm shop
```

Experiment: change a string in `api.py` and rebuild. The output shows `CACHED` for the `uv sync` step; only the last `COPY` reruns. Then change `pyproject.toml` (add a comment) and watch the dependency layer rebuild.

Experiment: `docker history tinyshop:0.1.0` shows every layer and its size.

Step 5 • Compose: API + PostgreSQL

TinyShop doesn't use a database yet (M3), but you'll set up the database container now so the stack is ready.

First, the init script that creates a second, empty database for tests. Create `deploy/postgres/init.sql`:

```
-- Runs once, when the database volume is first created.
CREATE DATABASE tinyshop_test;
```

Then `compose.yaml`:

```
services:
  api:
    build: . # build the Dockerfile in this folder
    ports:
      - "8000:8000" # host:container
    environment:
      # the host is `db`: the service name below. The app starts using this in M3.
      DATABASE_URL: postgresql://tinyshop:${POSTGRES_PASSWORD:-tinyshop}@db:5432/tinyshop
    depends_on:
      db:
        condition: service_healthy # wait until the db healthcheck passes, not just "started"
    restart: unless-stopped

  db:
    image: postgres:16 # official image, pinned major version
    environment:
      POSTGRES_USER: tinyshop
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-tinyshop} # from .env or your shell; default for
local dev
      POSTGRES_DB: tinyshop
    ports:
      - "5432:5432" # so tools on your laptop (psql, tests) can connect
    volumes:
      - pgdata:/var/lib/postgresql/data # named volume: data
survives restarts
      - ./deploy/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro # bind mount, read-
only
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U tinyshop -d tinyshop"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  pgdata:
```

`${POSTGRES_PASSWORD:-tinyshop}` means: use the variable `POSTGRES_PASSWORD` if it's set, otherwise `tinyshop`. Compose automatically reads a `.env` file next to `compose.yaml`, which is why `.env` is in `.gitignore`. The default is fine for a laptop; real environments set a real secret (M6, M13).

Named volume vs bind mount: `pgdata` is managed by Docker and survives `docker compose down`; `./deploy/...` maps a file from your repository into the container.

Add these targets to your `Makefile` (TABs!):

```
up: ## start the stack in the background
  docker compose up -d --build

down: ## stop the stack (keeps data)
  docker compose down

db: ## start only the database
  docker compose up -d db
```

Run it:

```
make up
docker compose ps          # both running; db "healthy", api "healthy" after ~15 s
docker compose logs -f api  # follow the API logs; Ctrl+C to stop following
curl -s localhost:8000/health
docker compose exec db psql -U tinyshop -d tinyshop -c '\l'      # list databases: tinyshop_test
exists
make down
```

Experiment (volumes): `docker compose down` and `make up` again: the database keeps its data. `docker compose down -v` deletes the volume too, and the init script runs again on the next start.

Experiment (networking): `docker compose exec api python -c "import socket; print(socket.gethostbyname('db'))"` shows the database's IP on the Compose network. Try `localhost` instead and see that it's the API container itself.

Run `make check M=02`.

Definition of done

- ☐ `make check M=02` is green.
- ☐ `make up` starts both containers; `docker compose ps` shows them healthy; `/docs` works at `localhost:8000`.
- ☐ Your image is several times smaller than `tinyshop:naive`, and `whoami` prints `app`.
- ☐ You can explain every line of the Dockerfile.
- ☐ Clean up: `docker image rm tinyshop:naive`; commit and push (`git commit -m "M2: Docker and Compose"`).

Interview questions

1. **Image vs container?** An image is a read-only template made of layers; a container is a running instance of it with its own writable layer.
2. **How do you make Docker builds fast?** Order instructions from least to most frequently changing, copy dependency files and install before copying code, use `.dockerignore`, and use BuildKit cache mounts in CI.
3. **Why multi-stage builds?** Build tools stay in the build stage; the runtime image is smaller, starts faster and has a smaller attack surface.
4. **Why not run as root in a container?** A compromised process as root has far more power to escape isolation or damage mounted data. Use a dedicated user; Kubernetes can enforce `runAsNonRoot`.
5. **Why can't the API reach the database at `localhost:5432` in Compose?** `localhost` is the API container itself. Containers on a Compose network reach each other by service name (`db`).
6. **How do you pass secrets to a container?** At runtime, through environment variables or mounted files from a secret store, never in the image. Anyone who can pull an image can read all its layers.

7. **What does `depends_on` with `service_healthy` guarantee, and what not?** Start order until the healthcheck passes. It doesn't protect against the database going away later, so the app still needs retries and a readiness check.
8. **Exec form vs shell form of CMD?** The exec form runs your process as PID 1 so it receives SIGTERM and can shut down gracefully; the shell form wraps it in `/bin/sh`, which doesn't forward the signal.

M3 • PostgreSQL

Time: ~1.5 weeks · **Tests:** `make check M=03` (no database needed) and `make test-db` (with PostgreSQL)
· **You'll have:** TinyShop storing everything in PostgreSQL, with versioned migrations and orders that stay correct even when two customers buy the last item at the same moment.

SQL is one of the most asked-for skills in every engineering job, and PostgreSQL is the database you'll meet most often; with pgvector (M9) it's also your vector database. This module is also where your first real bug class appears: race conditions. Nightshift will later diagnose slow queries and connection problems in exactly this database.

Concepts

TABLES, KEYS AND CONSTRAINTS

A **table** has typed **columns** and holds **rows**. Every table gets a **primary key** (a unique id). A **foreign key** says "this column must match a row in another table", so an order item can't point to a product that doesn't exist.

TinyShop's schema:

products		order_items		orders	
id	PK	product_id	PK, FK	id	PK
sku	UNIQUE	order_id	PK, FK	status	
name		quantity	1-100	total_cents	
price_cents	> 0	unit_price_cents	> 0	created_at	
stock	>= 0				
created_at					
(one product appears in many order items; one order contains one or more items)					

`order_items` exists because an order has *many* products and a product appears in *many* orders (many-to-many). Storing a list of products inside `orders` would break the rules of **normalisation**: every fact is stored once, in one place.

Constraints are your last line of defence. Pydantic validates requests, but data also arrives through scripts, migrations and other services. `CHECK (stock >= 0)`, `UNIQUE`, `NOT NULL` and foreign keys make invalid data impossible, whichever code writes it.

Useful types: `BIGINT GENERATED ALWAYS AS IDENTITY` (auto-increment ids), `TEXT`, `INTEGER`, `TIMESTAMPTZ` (timestamp **with** time zone; always use it), `BOOLEAN`, `JSONB`.

INDEXES

Without an index, finding rows means reading the whole table (a *sequential scan*). An index (a B-tree by default) is like a book's index: it finds rows in $O(\log n)$. Primary keys and `UNIQUE` columns get an index automatically. **Foreign keys don't**, which is a classic cause of slow joins and deletes. Every index also slows writes a little and uses disk, so you add them for real queries, and check with `EXPLAIN ANALYZE`.

TRANSACTIONS (ACID)

A transaction groups statements so they happen **all or nothing**:

```
BEGIN;
UPDATE products SET stock = stock - 2 WHERE id = 1;
INSERT INTO orders (total_cents) VALUES (2598);
COMMIT;      -- or ROLLBACK; to undo everything since BEGIN
```

Atomic (all or nothing), **C**onsistent (constraints hold), **I**solated (concurrent transactions don't see each other's half-done work), **D**urable (after COMMIT it survives a crash).

THE RACE CONDITION

Two customers, one mug left, both requests arrive at the same moment:

Time	Request A	Request B
1	reads stock = 1 ✓ enough	
2		reads stock = 1 ✓ enough
3	stock = 0, order created	
4		stock = -1?! order created

Both checks passed because both read *before* either wrote. This bug is called *check-then-act*, and it doesn't show up in normal tests, only under real concurrent load.

The fix: lock the rows you're about to change. `SELECT ... FOR UPDATE` locks the selected rows until your transaction ends; the second request **waits** at step 2, then reads the updated stock (0) and fails correctly. Two more rules:

- **Lock in a consistent order** (`ORDER BY id`). If A locks mug-then-shirt while B locks shirt-then-mug, each waits for the other forever: a **deadlock** (PostgreSQL detects it and kills one transaction).
- **Keep transactions short.** A lock held during a slow network call blocks everyone.

(Alternatives worth knowing for interviews: an atomic `UPDATE products SET stock = stock - 1 WHERE id = 1 AND stock >= 1` and check the affected row count; or *optimistic locking* with a version column. PostgreSQL's default isolation level is *Read Committed*; *SERIALIZABLE* also prevents this, at the cost of retries.)

SQL INJECTION AND PARAMETERS

Never build SQL with f-strings from user input:

```
conn.execute(f"SELECT * FROM products WHERE sku = '{sku}'")      # NEVER: sku = "x' OR '1'='1"
conn.execute("SELECT * FROM products WHERE sku = %s", (sku,))    # always: the driver sends data
separately
```

MIGRATIONS

The schema changes over time (new columns, new indexes), and every environment (your laptop, CI, staging, prod) must get the same changes, in the same order, exactly once. **Migrations** are numbered SQL files; a table `schema_migrations` records which ones have run. Rules: never edit a migration that has already run somewhere, add a new one instead; make changes backwards-compatible so old and new app versions can run side by side during a rolling deploy (M6). In companies you'll see tools like Alembic, Flyway or Liquibase; you'll write a tiny version yourself so you know what they do.

Step 1 • SQL warm-up in psql

```
make db                                # just the database (M2)
docker compose exec db psql -U tinyshop -d tinyshop  # an interactive SQL prompt
```

Type these one by one and read the results (`\q` quits, `\dt` lists tables, `\d name` describes one):

```
CREATE TABLE fruit (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                     name TEXT NOT NULL UNIQUE, price_cents INTEGER CHECK (price_cents > 0));
INSERT INTO fruit (name, price_cents) VALUES ('apple', 50), ('pear', 80), ('kiwi', 30);
SELECT * FROM fruit;
SELECT name FROM fruit WHERE price_cents < 60 ORDER BY price_cents DESC;
SELECT count(*), sum(price_cents), avg(price_cents) FROM fruit;
UPDATE fruit SET price_cents = 60 WHERE name = 'apple' RETURNING *;
INSERT INTO fruit (name, price_cents) VALUES ('apple', 10);    -- UNIQUE violation
INSERT INTO fruit (name, price_cents) VALUES ('plum', -5);    -- CHECK violation
BEGIN; DELETE FROM fruit; SELECT count(*) FROM fruit; ROLLBACK; SELECT count(*) FROM fruit;
DROP TABLE fruit;
```

Feel a row lock with two terminals, both in psql:

```
-- setup, in terminal A:
CREATE TABLE mug (id int PRIMARY KEY, stock int); INSERT INTO mug VALUES (1, 1);
-- A:
BEGIN; SELECT stock FROM mug WHERE id = 1 FOR UPDATE;
-- B: (this HANGS: it's waiting for A's lock)
BEGIN; SELECT stock FROM mug WHERE id = 1 FOR UPDATE;
-- A:
UPDATE mug SET stock = stock - 1 WHERE id = 1; COMMIT;
-- B now continues and sees stock = 0. Then:
ROLLBACK; DROP TABLE mug;
```

Now repeat without `FOR UPDATE`: B doesn't wait and reads the stale value. That's the race.

Step 2 • The first migration

```
mkdir migrations
```

Create `migrations/001_init.sql`. Here is the first table as an example; write `orders` and `order_items` yourself:

```
CREATE TABLE products (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  sku         TEXT NOT NULL UNIQUE,  
  name        TEXT NOT NULL,  
  price_cents INTEGER NOT NULL CHECK (price_cents > 0),  
  stock       INTEGER NOT NULL CHECK (stock >= 0),  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

- **orders**: `id` (identity primary key), `status` (text, default 'created', only 'created' or 'cancelled' allowed: a `CHECK (status IN (...))`), `total_cents` (integer ≥ 0), `created_at` (timestamptz, default now).
- **order_items**: `order_id` (references `orders(id)`, ON DELETE CASCADE), `product_id` (references `products(id)`), `quantity` (1-100: `CHECK (quantity BETWEEN 1 AND 100)`), `unit_price_cents` (> 0); the primary key is the pair (`order_id`, `product_id`).
- An index on `orders (created_at)`: reports ask "orders in the last hour" all the time.

Then `migrations/002_order_items_product_idx.sql`, because foreign keys aren't indexed automatically:

```
CREATE INDEX order_items_product_id_idx ON order_items (product_id);
```

Try your SQL in psql first; errors are much easier to read there.

Step 3 • psycopg in five minutes

```
uv add "psycopg[binary]"
```

```
import psycopg  
from psycopg.rows import dict_row  
  
url = "postgresql://tinyshop:tinyshop@localhost:5432/tinyshop"  
  
with psycopg.connect(url, row_factory=dict_row) as conn:  # rows come back as dicts  
    row = conn.execute("SELECT id, sku FROM products WHERE id = %s", (1,)).fetchone()  # dict or  
    None  
    rows = conn.execute("SELECT id FROM products WHERE id = ANY(%s)", ([1, 2, 3],)).fetchall()  
    new = conn.execute("INSERT INTO products (sku, name, price_cents, stock) "  
                        "VALUES (%(sku)s, %(name)s, %(price_cents)s, %(stock)s) RETURNING id",  
                        {"sku": "A-001", "name": "A", "price_cents": 100, "stock": 1}).fetchone()  
    with conn.transaction():  # a block that commits at the end, or rolls back on an  
        exception  
    ...  
# leaving the `with connect(...)` block: COMMIT if no exception, ROLLBACK if there was one; then  
close
```

- `%s` placeholders take a tuple; `%(name)s` placeholders take a dict (handy with `model.model_dump()`).
- `RETURNING` gives you back the inserted or updated row, including generated ids and defaults.

- Database errors are exceptions: a UNIQUE violation is `psycopg.errors.UniqueViolation`; all constraint errors are subclasses of `psycopg.errors.IntegrityError`.
- **The autocommit trap:** a normal connection automatically opens a transaction at the first statement. A `with conn.transaction():` block inside that open transaction becomes a *savepoint*, not a real commit. So if you want each block to commit on its own (you do, in `migrate`), open the connection with `autocommit=True`; then every `conn.transaction()` block is a real BEGIN ... COMMIT.
- Opening a connection per call is fine for now. In M6 you'll see why production apps use a **connection pool** (a classic incident: "too many connections").

Step 4 • The migration runner: `tinystore/db.py`

```
from pathlib import Path

MIGRATIONS_DIR = Path(__file__).resolve().parent.parent / "migrations"

def migrate(dsn: str, migrations_dir: Path = MIGRATIONS_DIR) -> list[str]:
    """Apply every *.sql file in `migrations_dir` that hasn't been applied yet, in name order.

    - Create the table `schema_migrations` (version TEXT PRIMARY KEY, applied_at TIMESTAMPTZ
      NOT NULL DEFAULT now())` if it doesn't exist.
    - Skip files whose name is already in schema_migrations.
    - Apply each new file AND record its name in ONE transaction, so a failing file leaves
      no half-applied changes and is not recorded. Files before it stay applied.
    - Return the names applied in this call, e.g. ["001_init.sql", "002_...sql"]; [] if none.
    """
```

The tests check all four rules, including what happens when the second file is broken.

`sorted(path.glob("*.sql"))` gives the files in order.

Step 5 • The PostgreSQL repository: `tinystore/pg_repository.py`

Write class `PostgresRepository` with `__init__(self, dsn: str)` and **exactly the same methods and behaviour** as `InMemoryRepository`. `tests/test_m03_postgres_repository.py` runs the same contract tests from M1 against it, plus two concurrency tests.

Method by method:

- **add_product:** `INSERT ... RETURNING id, sku, name, price_cents, stock; build a Product(**row)`. Catch `psycopg.errors.UniqueViolation` and raise `ConflictError` instead: the API must not know which database is behind it.
- **get_product, list_products:** `SELECT` with `WHERE id = %s`, or `ORDER BY id LIMIT %s OFFSET %s`.
- **place_order:** the important one. In **one transaction**: 1. lock the products: `SELECT ... FROM products WHERE id = ANY(%s) ORDER BY id FOR UPDATE`; 2. any id missing from the result → `NotFoundError`; any stock too low → `OutOfStockError` (raising inside the transaction rolls it back, so nothing changes);

3. insert the order with the total and `RETURNING id, status, total_cents, created_at`; 4. for each item: decrease stock and insert the `order_items` row with the current price; 5. return the `Order`.

- **get_order**: the order row plus its items (`ORDER BY product_id`); `NotFoundError` if missing.
- **cancel_order**: in one transaction, lock the order row (`SELECT status FROM orders WHERE id = %s FOR UPDATE`); missing → `NotFoundError`; already cancelled → `ConflictError`; put the stock back and set the status. One statement can restock every product of the order: `sql UPDATE products p SET stock = p.stock + oi.quantity FROM order_items oi WHERE oi.order_id = %s AND oi.product_id = p.id`; Why lock the order row? Two simultaneous cancels would otherwise both see `created` and restock twice.
- **top_products**: one query with two `JOIN`s (`order_items` → `orders`, `order_items` → `products`), `WHERE o.status = 'created'`, `GROUP BY p.sku, SUM(oi.quantity * oi.unit_price_cents)`, `ORDER BY revenue DESC, p.sku, LIMIT %s`. `SUM` returns a `bigint` / `numeric`; add `::int` to get a plain `int`.
- **ping**: `SELECT 1`.

Hints

- A helper `def _connect(self): return psycopg.connect(self.dsn, row_factory=dict_row)` keeps every method short.
- `with self._connect() as conn, conn.transaction():` opens a connection and a transaction in one line.
- In `place_order`, build `wanted = {item.product_id: item.quantity for item in data.items}` and pass `list(wanted)` to `ANY(%s)`. Turn the locked rows into `{row["id"]: row}` to look them up.
- To load an order inside another method's transaction, write a private `_load_order(conn, order_id)` that takes the open connection, and use it from both `get_order` and `cancel_order`.

Step 6 • Wire it up: `tinychop/main.py`

```
def build_repository(env: Mapping[str, str]) -> Repository:
    """DATABASE_URL set and not blank -> run migrate(url), return PostgresRepository(url).
    Otherwise -> InMemoryRepository(). (Import the Postgres modules inside the function, so
    the app still starts without psycopg installed.)"""

app = create_app(build_repository(os.environ))
```

Running migrations at start-up is fine for one instance. With several replicas in Kubernetes, you'll move them into a separate Job (M6); another interview topic.

Add migrations to the image. In the `Dockerfile` builder stage, after `COPY tinychop ./tinychop`:

```
COPY migrations ./migrations
```

Step 7 • Test against the real database

Add to the `Makefile`:

```
test-db:          ## PostgreSQL tests against the tinyshop_test database (make db first)
    TEST_DATABASE_URL=postgresql://tinyshop:tinyshop@localhost:5432/tinyshop_test uv run pytest -
q -m postgres
```

These tests wipe the database they're given, which is why they use `tinyshop_test` (created by your init script in M2) and never `tinyshop`. If `tinyshop_test` is missing because your volume existed before the init script, run `docker compose down -v` and `make db` again.

```
make check M=03    # file checks + wiring (no database needed); Postgres tests show as skipped
make db
make test-db       # contract tests, migrations and the race tests against real PostgreSQL
```

Break it on purpose: remove `FOR UPDATE` from `place_order` and run `make test-db` a few times.

`test_two_customers_race_for_the_last_item` fails: you've reproduced a real concurrency bug. Put it back.

Step 8 • Run the whole stack

```
make up
curl -s -X POST localhost:8000/products -H 'content-type: application/json' \
  -d '{"sku":"MUG-001","name":"Mug","price_cents":1299,"stock":10}'
curl -s -X POST localhost:8000/orders -H 'content-type: application/json' \
  -d '{"items":[{"product_id":1,"quantity":2}]}'
docker compose restart api && sleep 5
curl -s localhost:8000/products/1          # still there, stock 8: it's in PostgreSQL now
curl -s localhost:8000/ready
docker compose stop db && sleep 2
curl -s -i localhost:8000/ready            # 503: readiness sees the database is gone
curl -s -i localhost:8000/health          # 200: liveness doesn't care (M1)
docker compose start db
```

Look inside:

```
docker compose exec db psql -U tinyshop -d tinyshop
```

```
\dt
SELECT * FROM schema_migrations;
EXPLAIN ANALYZE SELECT * FROM orders WHERE created_at > now() - interval '1 hour';
```

With only a few rows, PostgreSQL chooses a sequential scan: reading a tiny table is faster than using the index. Add 100,000 old orders, one per minute going back about 70 days, so "the last hour" is a small slice:

```
INSERT INTO orders (total_cents, created_at)
  SELECT 100, now() - g * interval '1 minute' FROM generate_series(1, 100000) AS g;
ANALYZE orders;                                -- refresh the planner's statistics
EXPLAIN ANALYZE SELECT * FROM orders WHERE created_at > now() - interval '1 hour';
-- Index Scan using orders_created_at_idx ... Execution Time: ~0.05 ms
DROP INDEX orders_created_at_idx;
EXPLAIN ANALYZE SELECT * FROM orders WHERE created_at > now() - interval '1 hour';
-- Seq Scan on orders ... Execution Time: ~5 ms (about 100 times slower, and it grows with the
table)
CREATE INDEX orders_created_at_idx ON orders (created_at);
```

(If you had inserted the rows with the default `created_at = now()`, every row would be in the last hour and a sequential scan would be the right choice, even with the index. The planner picks by estimated cost, not by rule.) You'll use exactly this in M4 to create the "slow query" incident. Clean up with `docker compose down -v` when you're done experimenting.

Definition of done

- ☐ `make check M=03` and `make test-db` are green, including both race tests.
- ☐ `make up:orders` survive `docker compose restart api; /ready` returns 503 while the database is stopped.
- ☐ You ran the two-terminal lock demo and the `EXPLAIN ANALYZE` experiment.
- ☐ All M0–M3 tests pass (`make test` plus `make test-db`); committed and pushed (`git commit -m "M3: PostgreSQL"`).

Interview questions

- What does ACID mean?** Atomicity, consistency, isolation, durability: see the Concepts section.
- Two users buy the last item at the same time. What goes wrong and how do you prevent it?**
Check-then-act race: both read stock 1, both sell. Lock the rows (`SELECT ... FOR UPDATE`) in a transaction, or use an atomic conditional `UPDATE ... WHERE stock >= n` and check the row count, or optimistic locking with a version column, or `SERIALIZABLE` with retries.
- What is a deadlock, and how do you avoid it?** Two transactions each wait for a lock the other holds. Always lock in the same order (e.g. by id), keep transactions short, and retry when the database aborts one.
- When do you add an index, and what does it cost?** For columns used in `WHERE`, `JOIN` and `ORDER BY` on large tables, confirmed with `EXPLAIN ANALYZE`. Costs: slower writes and extra storage. Foreign keys are not indexed automatically in PostgreSQL.
- How do you prevent SQL injection?** Parameterised queries; never string formatting with input. Plus least-privilege database users.

6. **How do you change a database schema without downtime?** Additive, backwards-compatible migrations (add a nullable column, deploy code that writes both, backfill, then switch reads, then drop the old column later), run once per environment, never edited after they've run.
7. **Why validate in both Pydantic and the database?** The API validates for good error messages; database constraints protect the data from every other writer and from bugs.
8. **Why do the same tests run against the in-memory and the PostgreSQL repository?** Contract tests prove both implementations behave the same, so the fast in-memory version is a trustworthy stand-in in unit tests.